

Лабораторна робота № 1

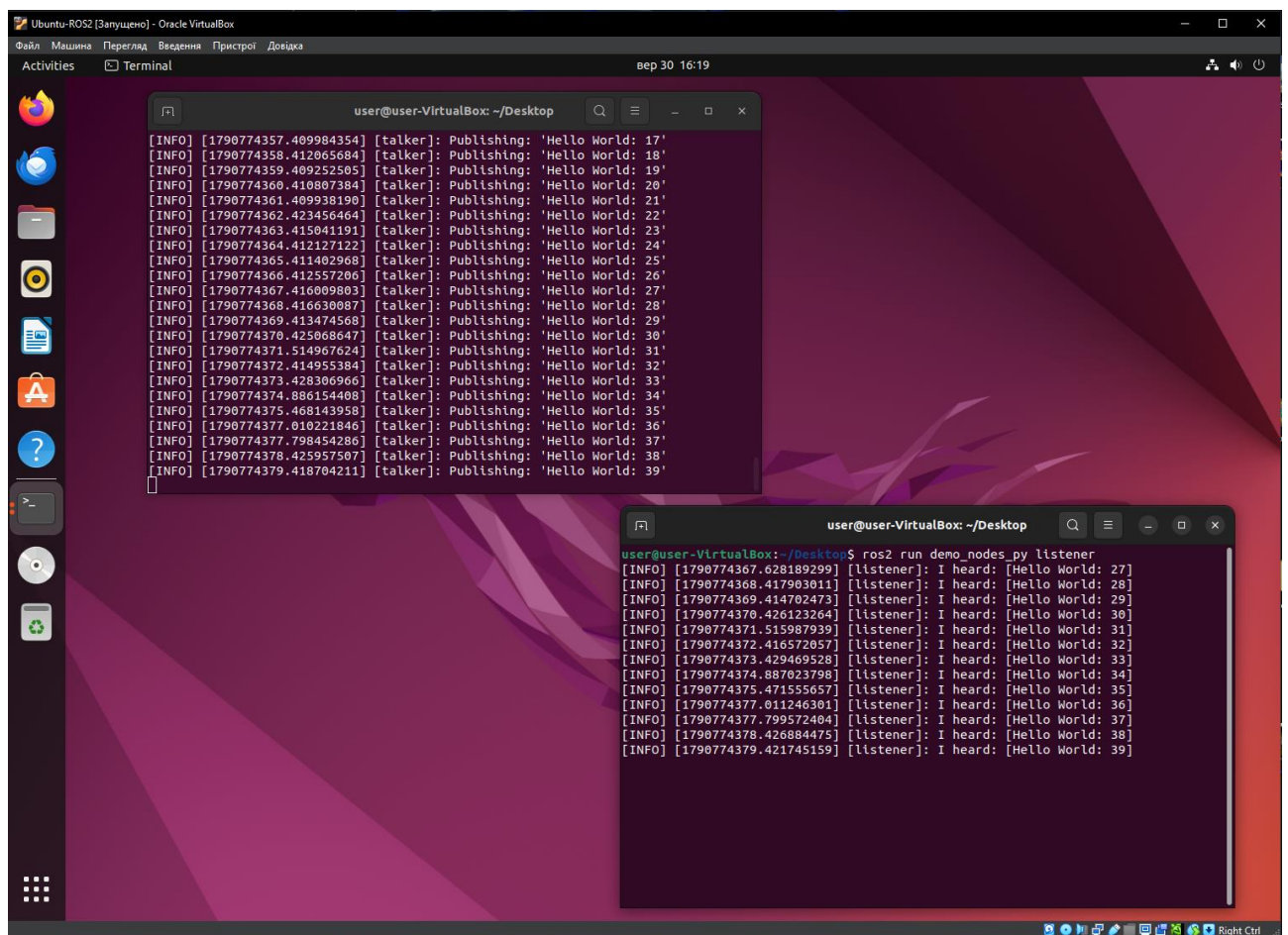
Мета роботи:

Сформувати розуміння архітектури ROS2, принципів роботи нод, топіків, механізму Publisher/Subscriber та навчитися створювати власний ROS2-пакет і реалізовувати обмін повідомленнями між нодами.

Хід роботи

Перевірка встановлення ROS2 та запуск прикладу turtlesim:

Для перевірки коректності встановлення ROS2 було виведено змінну середовища ROS_DISTRO та запущено демонстраційні ноди talker і listener. Нода talker надавала повідомлення Hello World: N, а listener їх успішно отримувала(рис. 1).



```
user@user-VirtualBox: ~/Desktop
[INFO] [1790774357.409984354] [talker]: Publishing: 'Hello World: 17'
[INFO] [1790774358.412065684] [talker]: Publishing: 'Hello World: 18'
[INFO] [1790774359.409222595] [talker]: Publishing: 'Hello World: 19'
[INFO] [1790774360.410807384] [talker]: Publishing: 'Hello World: 20'
[INFO] [1790774361.409938198] [talker]: Publishing: 'Hello World: 21'
[INFO] [1790774362.423456464] [talker]: Publishing: 'Hello World: 22'
[INFO] [1790774363.415041191] [talker]: Publishing: 'Hello World: 23'
[INFO] [1790774364.412127122] [talker]: Publishing: 'Hello World: 24'
[INFO] [1790774365.411402968] [talker]: Publishing: 'Hello World: 25'
[INFO] [1790774366.412557206] [talker]: Publishing: 'Hello World: 26'
[INFO] [1790774367.416009803] [talker]: Publishing: 'Hello World: 27'
[INFO] [1790774368.416630087] [talker]: Publishing: 'Hello World: 28'
[INFO] [1790774369.413474568] [talker]: Publishing: 'Hello World: 29'
[INFO] [1790774370.425068647] [talker]: Publishing: 'Hello World: 30'
[INFO] [1790774371.514967624] [talker]: Publishing: 'Hello World: 31'
[INFO] [1790774372.414955384] [talker]: Publishing: 'Hello World: 32'
[INFO] [1790774373.428306966] [talker]: Publishing: 'Hello World: 33'
[INFO] [1790774374.886154408] [talker]: Publishing: 'Hello World: 34'
[INFO] [1790774375.468143958] [talker]: Publishing: 'Hello World: 35'
[INFO] [1790774377.010221846] [talker]: Publishing: 'Hello World: 36'
[INFO] [1790774377.790454286] [talker]: Publishing: 'Hello World: 37'
[INFO] [1790774378.425957587] [talker]: Publishing: 'Hello World: 38'
[INFO] [1790774379.418704211] [talker]: Publishing: 'Hello World: 39'

user@user-VirtualBox: ~/Desktop
user@user-VirtualBox:~/Desktop$ ros2 run demo_nodes_py listener
[INFO] [1790774367.628189299] [listener]: I heard: [Hello World: 27]
[INFO] [1790774368.417903011] [listener]: I heard: [Hello World: 28]
[INFO] [1790774369.414702473] [listener]: I heard: [Hello World: 29]
[INFO] [1790774370.426123264] [listener]: I heard: [Hello World: 30]
[INFO] [1790774371.515987939] [listener]: I heard: [Hello World: 31]
[INFO] [1790774372.416572057] [listener]: I heard: [Hello World: 32]
[INFO] [1790774373.429469528] [listener]: I heard: [Hello World: 33]
[INFO] [1790774374.887023798] [listener]: I heard: [Hello World: 34]
[INFO] [1790774375.471555657] [listener]: I heard: [Hello World: 35]
[INFO] [1790774377.011246301] [listener]: I heard: [Hello World: 36]
[INFO] [1790774377.799572404] [listener]: I heard: [Hello World: 37]
[INFO] [1790774378.426884475] [listener]: I heard: [Hello World: 38]
[INFO] [1790774379.421745159] [listener]: I heard: [Hello World: 39]
```

Рисунок 1 - Перевірка встановлення ROS2 (talker / listener)

Далі було запущено симулятор turtlesim, у результаті відкрилося графічне вікно з черепахою. В окремому терміналі запущено ноду керування, яка зчитує натискання клавіш-стрілок і перетворює їх на команди руху для черепахи з клавіатури, було отримано траєкторію руху (рис. 2).

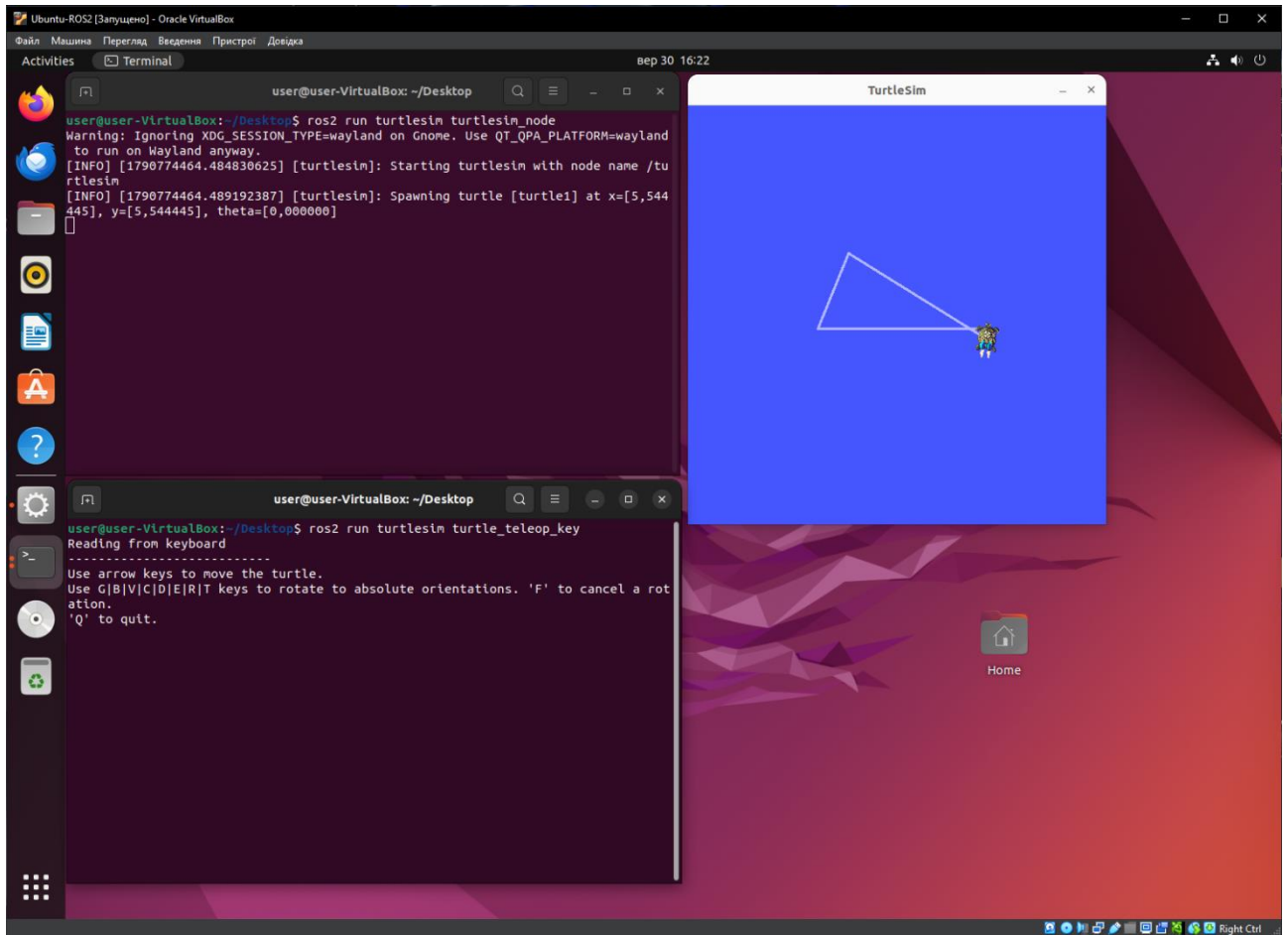


Рисунок 2 - Керування черепахою у turtlesim

За допомогою команди `ros2 node list` визначено активні ноди системи (рис. 3).

```
user@user-VirtualBox:~/Desktop$ ros2 node list
/teleop_turtle
/turtlesim
```

Рисунок 3 - Результат виконання команди `ros2 node list`

Командою `ros2 topic list -t` отримано перелік активних топіків та типи їх повідомлень (рис. 4).

```

user@user-VirtualBox:~/Desktop$ ros2 topic list -t
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]

```

Рисунок 4 - Результат виконання команди `ros2 topic list -t`

Для перегляду повідомлень було виконано команду `ros2 topic echo /turtle1/cmd_vel`. Під час натискання клавіш у терміналі у консоль виводилися повідомлення, що містять лінійну та кутову швидкості (рис. 5). Це демонструє, що нода `/teleop_turtle` є Publisher, а `/turtlesim` є Subscriber топіка `/turtle1/cmd_vel`.

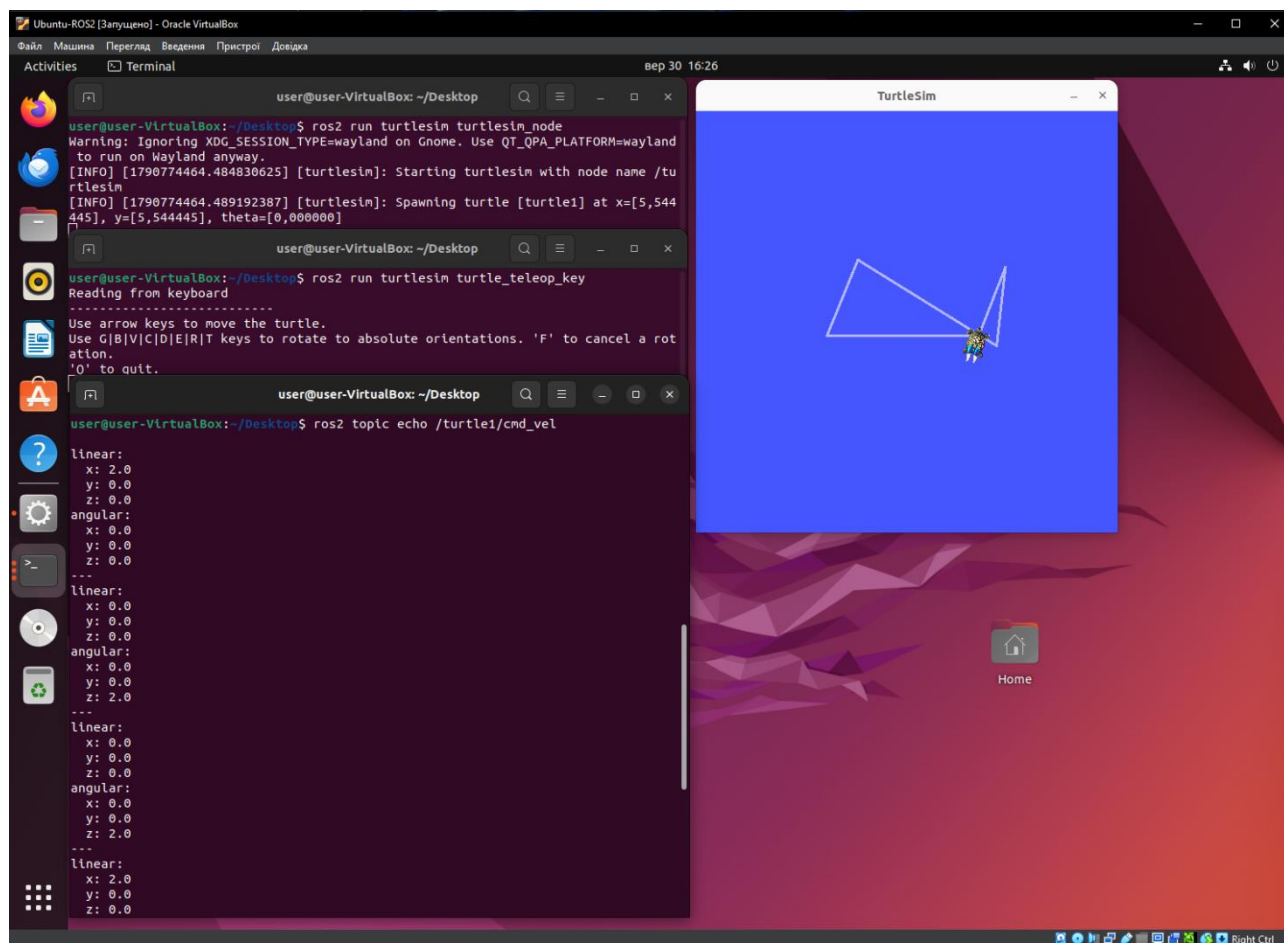


Рисунок 5 - Результат виконання команди `ros2 topic echo /turtle1/cmd_vel`

Створення власного пакету:

Створено робочий простір `~/ros2_ws` з каталогом `src`, у якому командою `ros2 pkg create --build-type ament_python --license Apache-2.0 --dependencies rclpy`

std_msgs --node-name publisher_node lab1_pubsub створено пакет lab1_pubsub з типом збірки ament_python та залежностями rclpy і std_msgs (рис. 6).

```
/home/user/ros2_ws/src/lab1_pubsub
├── lab1_pubsub
│   ├── __init__.py
│   └── publisher_node.py
├── LICENSE
├── package.xml
├── resource
│   └── lab1_pubsub
├── setup.cfg
├── setup.py
└── test
    ├── test_copyright.py
    ├── test_flake8.py
    └── test_pep257.py
```

Рисунок 6 - Структура пакету lab1_pubsub

Таблиця 1 - Призначення файлів пакету

Файл / каталог	Призначення
package.xml	Маніфест пакету
setup.py	Налаштування збірки. В entry_points зареєстровано виконувані ноди
setup.cfg	Вказує куди встановлювати виконувані файли, щоб їх знаходила команда ros2 run
resource/lab1_pubsub	Маркерний файл для індексу пакетів ament
lab1_pubsub/	Python-модуль з вихідним кодом нод
publisher_node.py	Нода-публікатор
subscriber_node.py	Нода-підписник
test/	Автоматичні тести коду

Робочий простір було зібрано командою colcon build, після чого середовище активовано командою source install/setup.bash. Коректність встановлення пакету перевірено командами ros2 pkg list, ros2 pkg prefix lab1_pubsub та ros2 pkg executables lab1_pubsub (рис. 7).

```
lab1_pubsub
/home/user/ros2_ws/install/lab1_pubsub
lab1_pubsub publisher_node
```

Рисунок 7 - Збірка робочого простору та перевірка наявності пакету

Реалізація Publisher:

Нода `lab1_publisher` реалізована у вигляді класу `Lab1Publisher`, успадкованого від `rclpy.node.Node`. У конструкторі створюється `Publisher` для `lab1_topic` з типом повідомлення `std_msgs/msg/String` та глибиною черги 10, а також таймер з періодом 1,0 с. У функції зворотного виклику таймера формується текстове повідомлення з порядковим номером, воно публікується у топик, а факт публікації фіксується в журналі на рівні `INFO`.

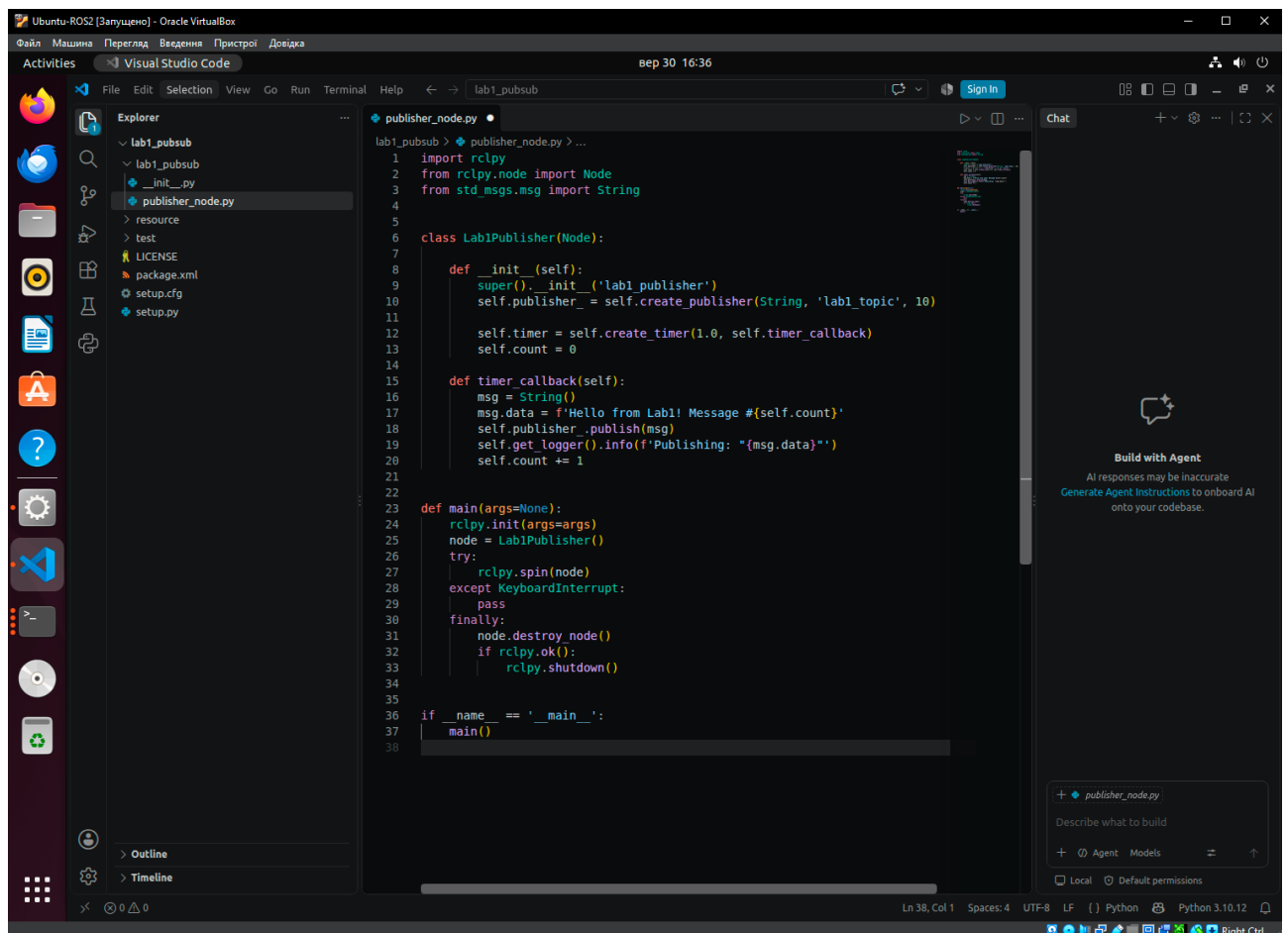


Рисунок 8 - Код ноди Publisher

Реалізація Subscriber:

Нода `lab1_subscriber` реалізована класом `Lab1Subscriber`. У конструкторі створюється підписка на `lab1_topic` того ж типу `std_msgs/msg/String`. При надходженні кожного повідомлення викликається функція `listener_callback`, яка виводить отриманий текст у консоль через логер ROS2.

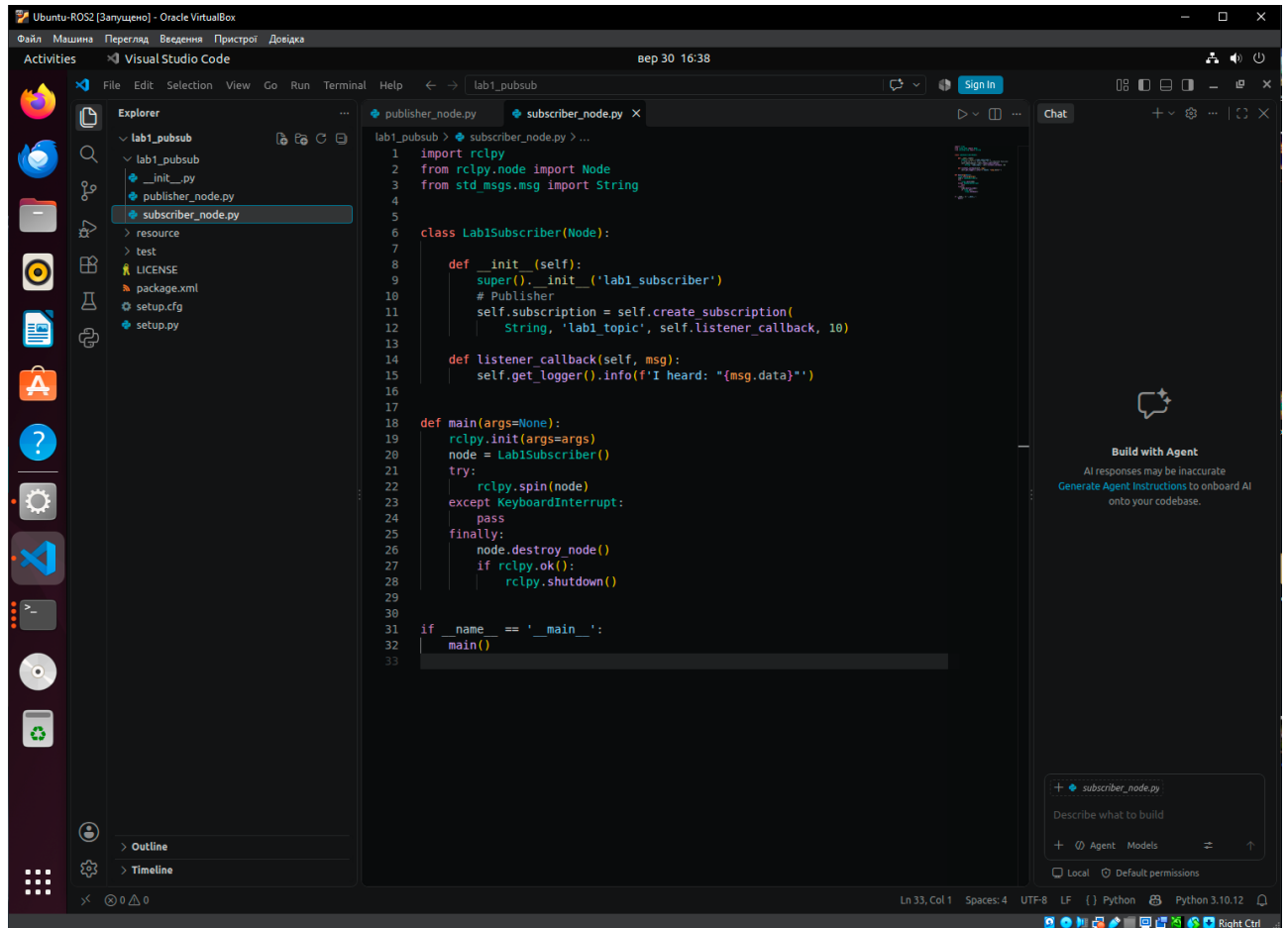


Рисунок 9 - Код ноди Subscriber

Для того щоб ноди можна було запускати командою `ros2 run`, їх зареєстровано у файлі `setup.py` (рис. 10).

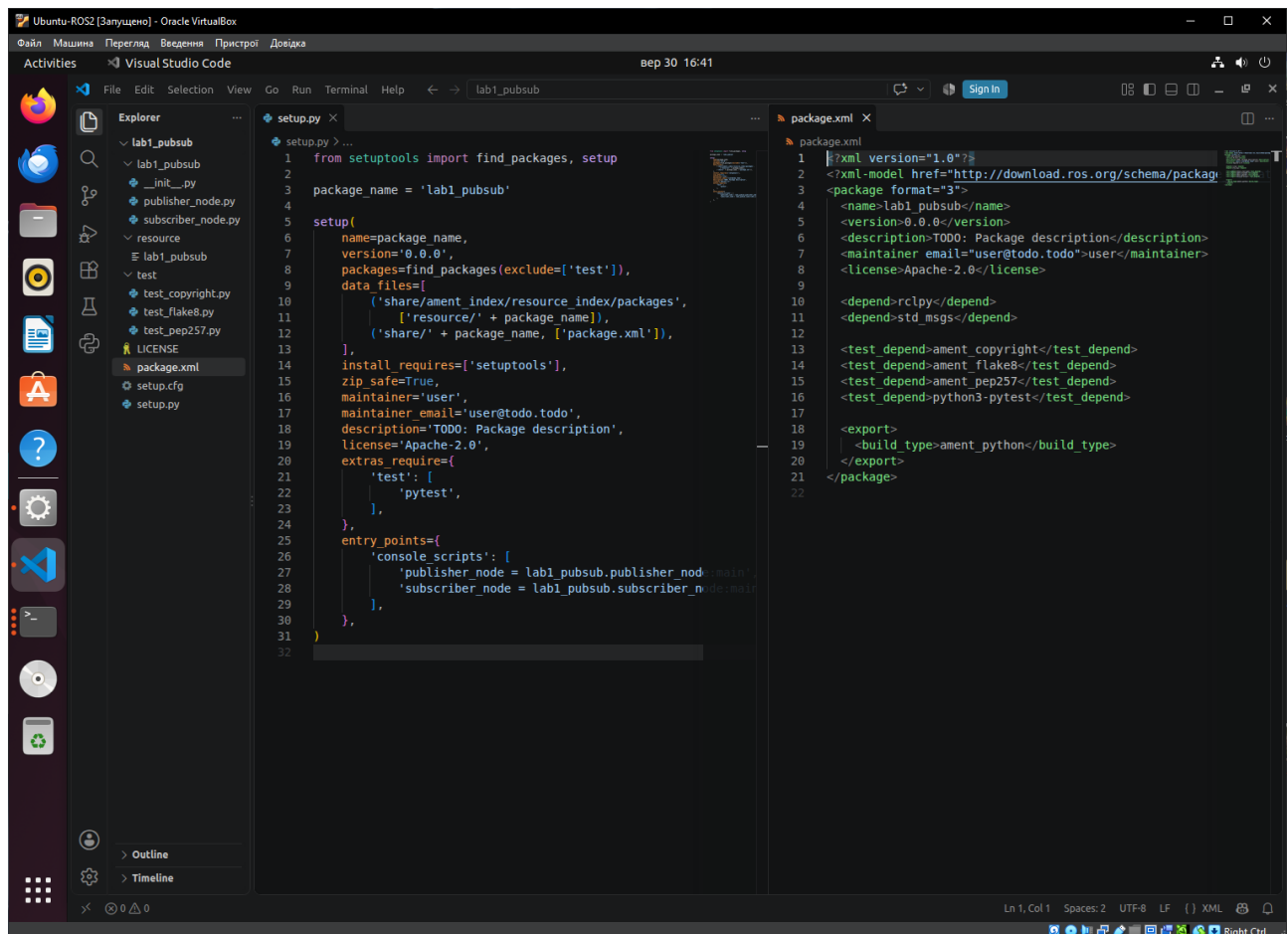


Рисунок 10 - Реєстрація нод у setup.py та залежності у package.xml

Перевірка роботи системи:

Publisher і Subscriber запущено в окремих терміналах командами `ros2 run lab1_pubsub publisher_node` та `ros2 run lab1_pubsub subscriber_node`. Publisher щосекунди публікує повідомлення та виводить "Hello from Lab1! Message #N", а Subscriber отримує ці повідомлення та виводить I heard: "Hello from Lab1! Message #N". Номери повідомлень збігаються, що свідчить про коректний обмін даними (рис. 11).

```
user@user-VirtualBox: ~/ros2_ws
[INFO] [1790775807.585218252] [lab1_publisher]: Publishing: "Hello from Lab1! Message #34"
[INFO] [1790775807.585764894] [lab1_subscriber]: I heard: "Hello from Lab1! Message #34"
[INFO] [1790775808.584429788] [lab1_publisher]: Publishing: "Hello from Lab1! Message #35"
[INFO] [1790775808.584632922] [lab1_subscriber]: I heard: "Hello from Lab1! Message #35"
[INFO] [1790775809.587458987] [lab1_publisher]: Publishing: "Hello from Lab1! Message #36"
[INFO] [1790775809.588366037] [lab1_subscriber]: I heard: "Hello from Lab1! Message #36"
[INFO] [1790775810.583662799] [lab1_publisher]: Publishing: "Hello from Lab1! Message #37"
[INFO] [1790775810.583913382] [lab1_subscriber]: I heard: "Hello from Lab1! Message #37"
[INFO] [1790775811.628159733] [lab1_publisher]: Publishing: "Hello from Lab1! Message #38"
[INFO] [1790775811.628159733] [lab1_subscriber]: I heard: "Hello from Lab1! Message #38"
[INFO] [1790775812.608883759] [lab1_publisher]: Publishing: "Hello from Lab1! Message #39"
[INFO] [1790775812.608883759] [lab1_subscriber]: I heard: "Hello from Lab1! Message #39"
[INFO] [1790775813.582463760] [lab1_publisher]: Publishing: "Hello from Lab1! Message #40"
[INFO] [1790775813.583622652] [lab1_subscriber]: I heard: "Hello from Lab1! Message #40"
[INFO] [1790775814.593728435] [lab1_publisher]: Publishing: "Hello from Lab1! Message #41"
[INFO] [1790775814.594001141] [lab1_subscriber]: I heard: "Hello from Lab1! Message #41"
[INFO] [1790775815.582450288] [lab1_publisher]: Publishing: "Hello from Lab1! Message #42"
[INFO] [1790775815.582939197] [lab1_subscriber]: I heard: "Hello from Lab1! Message #42"
[INFO] [1790775816.584233469] [lab1_publisher]: Publishing: "Hello from Lab1! Message #43"
[INFO] [1790775816.584233469] [lab1_subscriber]: I heard: "Hello from Lab1! Message #43"
[INFO] [1790775817.583178248] [lab1_publisher]: Publishing: "Hello from Lab1! Message #44"
[INFO] [1790775817.583178248] [lab1_subscriber]: I heard: "Hello from Lab1! Message #44"
[INFO] [1790775818.583781299] [lab1_publisher]: Publishing: "Hello from Lab1! Message #45"
[INFO] [1790775818.584045401] [lab1_subscriber]: I heard: "Hello from Lab1! Message #45"
[INFO] [1790775819.585781088] [lab1_publisher]: Publishing: "Hello from Lab1! Message #46"
[INFO] [1790775819.585994027] [lab1_subscriber]: I heard: "Hello from Lab1! Message #46"
[INFO] [1790775819.585994027] [lab1_publisher]: Publishing: "Hello from Lab1! Message #47"
[INFO] [1790775819.585994027] [lab1_subscriber]: I heard: "Hello from Lab1! Message #47"
```

Рисунок 11 - Одночасна робота нод Publisher та Subscriber

Відображення нод і топиків перевірено стандартними командами ROS2 (рис. 12).

```
user@user-VirtualBox: ~/Desktop
user@user-VirtualBox:~/Desktop$ ros2 node list
ros2 topic list -t
ros2 topic info /lab1_topic
/lab1_publisher
/lab1_subscriber
/teleop_turtle
/turtlesim
/lab1_topic [std_msgs/msg/String]
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
Type: std_msgs/msg/String
Publisher count: 1
Subscription count: 1
```

Рисунок 12 - Перевірка нод і топиків свого пакету

Команда `ros2 topic echo /lab1_topic` відобразила повідомлення, що передаються через топик, а `ros2 topic hz /lab1_topic` підтвердила частоту публікації ± 1 Гц, що відповідає вимозі завдання (рис. 13).


```
user@user-VirtualBox: ~/Desktop
data: 'Hello from Lab1! Message #78'
---
data: 'Hello from Lab1! Message #79'
---
data: 'Hello from Lab1! Message #80'
---
data: 'Hello from Lab1! Message #81'
---
data: 'Hello from Lab1! Message #82'
---
data: 'Hello from Lab1! Message #83'
---
data: 'Hello from Lab1! Message #84'
---
^Cuser@user-VirtualBox:~/Desktop$ ros2 topic hz /lab1_topic
average rate: 1.001
  min: 0.999s max: 1.000s std dev: 0.00065s window: 3
average rate: 1.000
  min: 0.999s max: 1.000s std dev: 0.00060s window: 5
average rate: 1.000
  min: 0.999s max: 1.001s std dev: 0.00076s window: 6
average rate: 1.000
  min: 0.999s max: 1.001s std dev: 0.00073s window: 8
average rate: 1.000
```

Рисунок 13 - Перегляд повідомлень топіка та перевірка частоти публікації

Висновок:

У ході виконання лабораторної роботи було вивчено архітектуру ROS2: система складається з незалежних нод, які взаємодіють між собою через іменовані топіки за схемою Publisher/Subscriber, не знаючи одна про одну безпосередньо.

На прикладі turtlesim було продемонстровано взаємодію двох нод: /teleop_turtle публікує команди швидкості в топік /turtle1/cmd_vel, а /turtlesim підписана на нього та рухає черепаху.

Було створено власний пакет lab1_pubsub з типом збірки ament_python, у якому реалізовано ноду-публікатор, що щосекунди надсилає повідомлення з логуванням на рівні INFO, та ноду-підписника, що отримує ці повідомлення й виводить їх у консоль через ROS2.